Анализ рынка вакансий Java: RustFS, MapReduce и DuckDB

Домашнее задание №3. Вариант: Java. Данные: it_vacancies_2024.csv (229 388 вакансий, 848 МБ). Все результаты получены на моём компьютере (Windows 11, 16 логических ядер, 31 ГБ ОЗУ, Python 3.11, DuckDB 1.5.6, RustFS в Docker); исходный код — в репозитории (каталог hw3/).

Критерий варианта

Вакансия относится к варианту «Java», если в её названии есть слово java (без учёта регистра, границы слова) и нет javascript / java script. Получилось 7 206 вакансий, у 4 870 указаны ключевые навыки, у 1 386 — зарплата в RUR (у 1 074 — и навыки, и зарплата). Одно и то же правило задано в Python (vac.py) и независимо в SQL (duck.py). Навык = элемент поля key_skills (разделитель «;», обрезка пробелов, дубликаты внутри вакансии удалены, регистр сохранён). Зарплата = середина вилки (или единственная граница), только RUR, gross переводится в net (×0,87), значения вне 10 000…1 500 000 отбрасываются.

1–2. Объектное хранилище и загрузка данных

RustFS запущен в Docker (4 «диска» из ДЗ №2, порт API 9000, консоль 9001). Создан бакет hh-lake, в него загружен исходный CSV (multipart, 64 МБ). Целостность: размер объекта в бакете совпадает с локальным файлом — 847 619 384 байт.

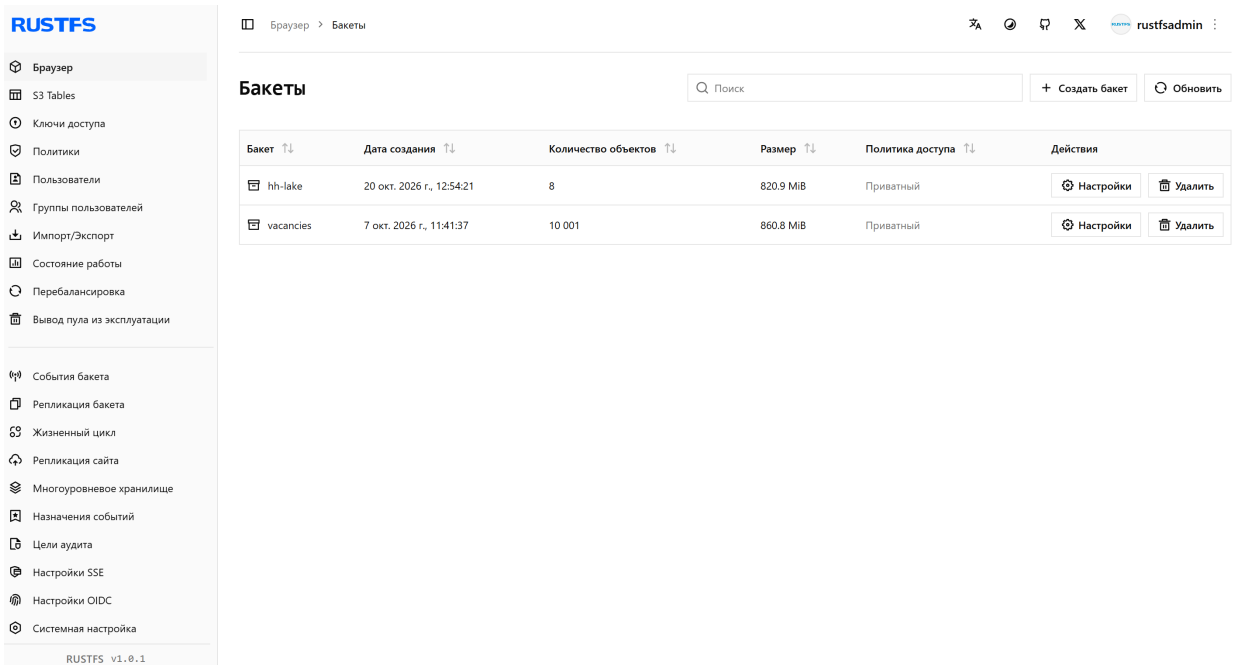
```
CONTAINER ID   IMAGE                  STATUS             PORTS                                                                   NAMES
993d957f2d5a   rustfs/rustfs:latest   Up About an hour   0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp   rustfs
```

```
bucket created: hh-lake
uploaded in 13.9s
--- list_objects_v2 ---
dataset/it_vacancies_2024.csv 847619384 2026-10-20 09:56:18.597000+00:00
local md5: 45041a115287e29b41fc8f82818ec5a6
```

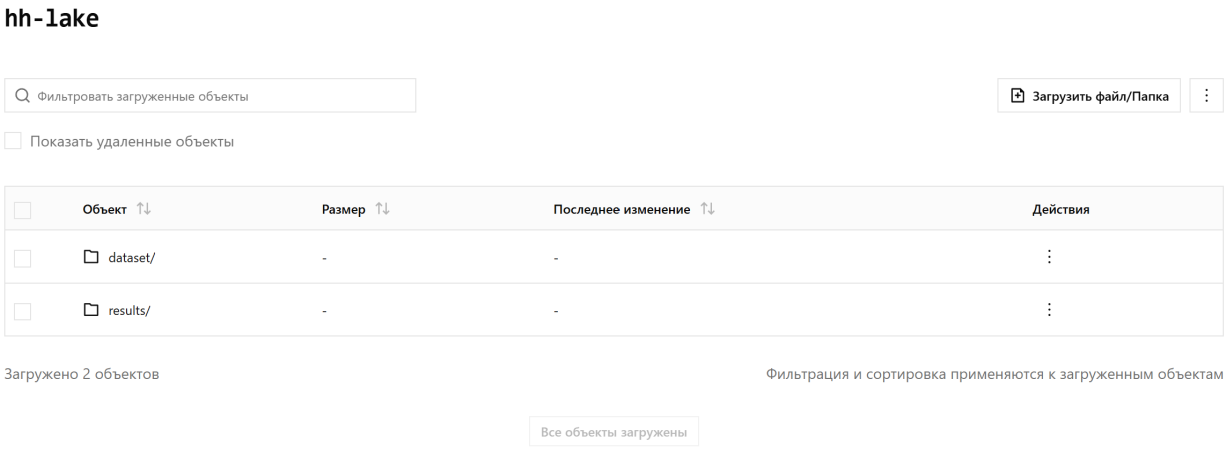
Листинг бакета после всех этапов (результаты MapReduce и DuckDB лежат в нём же):

```
--- list_objects_v2(Bucket='hh-lake') ---
dataset/it_vacancies_2024.csv                     847619384  2026-10-20 09:56:18
results/duckdb/skills_k1.csv                          35368  2026-10-20 10:28:16
results/duckdb/skills_k2.csv                         696232  2026-10-20 10:28:17
results/duckdb/skills_k3.csv                        5781511  2026-10-20 10:28:18
results/mapreduce/skills_k1.csv                       36739  2026-10-20 10:27:45
results/mapreduce/skills_k2.csv                      716801  2026-10-20 10:27:46
results/mapreduce/skills_k3.csv                     5917225  2026-10-20 10:27:47
results/mapreduce/stats.json                            127  2026-10-20 10:27:48
```

```
PS C:\Users\ninak\PycharmProjects\eng_dan> docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED        STATUS                  PORTS                                                                
```



Скриншот: консоль RustFS (localhost:9001), бакет hh-lake



Скриншот: объекты бакета hh-lake (dataset/ и results/)

3. Подсчёт навыков, пар и троек двумя способами

3.1 Собственный MapReduce (mapreduce.py)

Чтение из хранилища чанками. Файл не копируется на диск: драйвер читает объект s3://hh-lake/dataset/it_vacancies_2024.csv последовательными Range-запросами (get_object(Range=bytes=a-b)). Описания вакансий содержат переводы строк внутри кавычек, поэтому резать CSV по границе байтового диапазона нельзя: драйвер склеивает диапазоны и отдаёт mapper-у блок из целых записей — граница выбирается на последнем переводе строки с чётным числом кавычек слева, неполный хвост переносится в следующий блок.

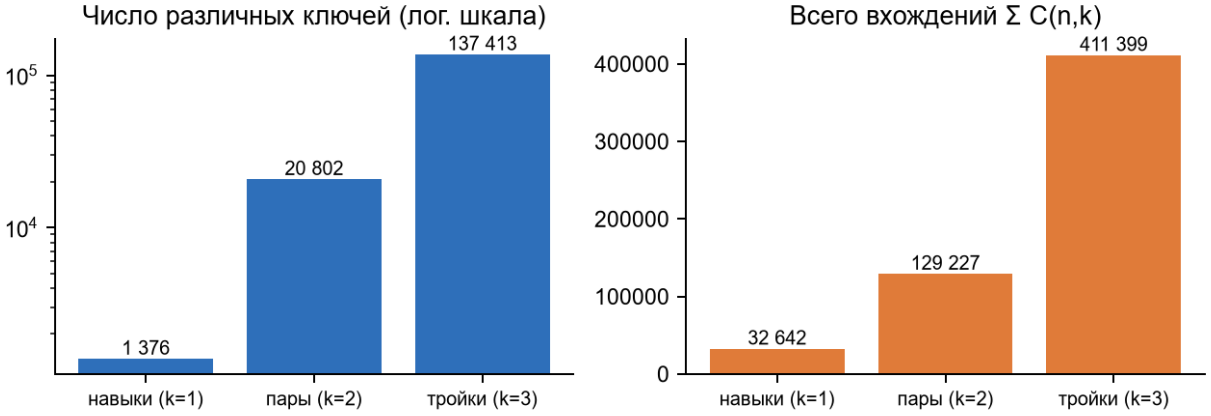
Map. Mapper разбирает блок (csv), оставляет Java-вакансии, строит отсортированный список уникальных навыков и порождает все сочетания размера 1, 2, 3 (itertools.combinations); для вакансий с зарплатой одновременно копит зарплаты по каждому ключу. Combiner: внутри mapper-а счётчики свёрнуты в Counter. Reduce: драйвер суммирует счётчики и объединяет списки зарплат; итог — таблицы навыков, пар, троек (частота, число зарплат, медиана) в CSV, которые загружены в бакет results/mapreduce/.

Отказ mapper-а. Флаг --fail-chunk 7 заставляет mapper упасть на 7-м чанке при первой попытке; драйвер перезапускает задачу (до 3 попыток). Итоговые числа не изменились (см. сверку ниже):

```
chunk 7: FAILED (injected mapper failure on chunk 7); retry 1
chunk 7: ok (attempt 1)
MODE=opt chunk=8MB workers=4 io_threads=8 chunks=102 retries=1 time=11.48s
stats: {'rows': 229388, 'java': 7206, 'java_with_skills': 4870, 'java_with_salary': 1386, 'java_skills_and_salary': 1074}
distinct: {'k1': 1376, 'k2': 20802, 'k3': 137413} total: {'k1': 32642, 'k2': 129227, 'k3': 411399}
```

3.2 Три размера «стека»: рост числа ключей

k	Что считаем	Различных ключей	Сумма частот	Σ C(n,k) по вакансиям
1	навыки	1 376	32 642	32 642
2	пары	20 802	129 227	129 227
3	тройки	137 413	411 399	411 399



вакансий с навыками: 4870 среднее число навыков: 6.70 макс: 30
n_skills quantiles: ([4.0, 6.0, 9.0, 12.0, 19.0],)
k=1: сумма C(n,1) по вакансиям = 32642
k=2: сумма C(n,2) по вакансиям = 129227
k=3: сумма C(n,3) по вакансиям = 411399
вакансий с >=15 навыками: 216 дают 43.9% всех троек

Почему растёт. Вакансия с n навыками даёт n навыков, C(n,2)=n(n−1)/2 пар и C(n,3)=n(n−1)(n−2)/6 троек. В среднем у вакансии 6,7 навыков (медиана 6, 90-й процентиль 12, максимум 30), поэтому сумма частот растёт в 4,0 раза от навыков к парам и в 3,2 раза от пар к тройкам, а число различных ключей — в 15 и 6,6 раза (комбинаций много, и большинство встречаются 1–2 раза: «длинный хвост»). Суммарное число вхождений совпадает с Σ C(n,k) до единицы (32 642 / 129 227 / 411 399) — это независимая проверка корректности подсчёта. Рост нелинеен по n: 216 вакансий с ≥15 навыками (4,4 % вакансий) порождают 43,9 % всех троек.

Топ-10 по каждому размеру (результат MapReduce)

Навыки	Вакансий	С зарплатой	Медиана, руб.
Java	4484	985	190 000

Навыки	Вакансий	С зарплатой	Медиана, руб.
Spring Framework	2210	550	179 175
PostgreSQL	1903	529	165 000
SQL	1449	336	200 000
Git	1179	367	175 000
Spring Boot	1032	230	230 000
Kafka	991	164	250 000
Docker	773	198	225 000
Hibernate ORM	691	281	65 250
Kotlin	636	141	250 000

Пары	Вакансий	С зарплатой	Медиана, руб.
Java + Spring Framework	2081	519	175 000
Java + PostgreSQL	1773	508	160 000
Java + SQL	1348	302	200 000
PostgreSQL + Spring Framework	1159	370	117 500
Git + Java	1091	338	177 500
Java + Spring Boot	967	220	237 500
Java + Kafka	922	150	250 000
Docker + Java	694	178	225 000
Git + Spring Framework	668	215	170 000
Hibernate ORM + Java	659	276	65 250

Тройки	Вакансий	С зарплатой	Медиана, руб.
Java + PostgreSQL + Spring Framework	1098	358	115 000
Git + Java + Spring Framework	628	203	170 000
Java + SQL + Spring Framework	565	149	225 000
Git + Java + PostgreSQL	552	222	170 000
Hibernate ORM + Java + Spring Framework	546	231	65 250
Java + Kafka + PostgreSQL	524	96	240 000
Java + Kafka + Spring Framework	510	75	240 000
Java + PostgreSQL + Spring Boot	493	116	250 000
Git + Java + SQL	477	139	200 000
Java + PostgreSQL + SQL	438	138	200 000

3.3 DuckDB: SQL-запрос к тем же данным в хранилище (duck.py, 03_duckdb.py)

DuckDB читает объект напрямую из RustFS через расширение httpfs (s3://hh-lake/dataset/it_vacancies_2024.csv). Условие варианта написано в SQL самостоятельно (regexp_matches по названию):

```
CREATE OR REPLACE TEMP TABLE vac AS
SELECT
  TRY_CAST(vacancy_id AS BIGINT) AS vacancy_id,
  area, name, key_skills, experience, schedule, status, employer_id, published_at, salary_from, salary_to, gross,
  CASE WHEN currency = 'RUR' THEN
     CASE WHEN s BETWEEN 10000 AND 1500000 THEN s END
  END AS sal
FROM (
  SELECT *,
    coalesce((TRY_CAST(salary_from AS DOUBLE) + TRY_CAST(salary_to AS DOUBLE)) / 2,
             TRY_CAST(salary_from AS DOUBLE), TRY_CAST(salary_to AS DOUBLE))
      * CASE WHEN lower(trim(gross)) = 'true' THEN 0.87 ELSE 1 END AS s
  FROM read_csv('s3://hh-lake/dataset/it_vacancies_2024.csv', header=true, all_varchar=true, quote='"', escape='"',
               max_line_size=33554432)
)
WHERE regexp_matches(name, '(?i)\bjava\b')
  AND NOT regexp_matches(name, '(?i)javascript|java\s+script')
```

```
CREATE OR REPLACE TEMP TABLE sk AS
SELECT DISTINCT vacancy_id, trim(skill) AS skill, sal, employer_id
FROM (SELECT vacancy_id, sal, employer_id, unnest(string_split(key_skills, ';')) AS skill FROM vac)
WHERE trim(skill) <> ''

SELECT a.skill AS skill_1, b.skill AS skill_2, count(*) AS vacancies, count(a.sal) AS n_salary,
       round(median(a.sal), 2) AS median_salary
FROM sk a JOIN sk b ON a.vacancy_id = b.vacancy_id AND a.skill < b.skill
GROUP BY 1, 2 ORDER BY vacancies DESC, skill_1, skill_2

SELECT a.skill AS skill_1, b.skill AS skill_2, c.skill AS skill_3, count(*) AS vacancies,
       count(a.sal) AS n_salary, round(median(a.sal), 2) AS median_salary
FROM sk a JOIN sk b ON a.vacancy_id = b.vacancy_id AND a.skill < b.skill
          JOIN sk c ON a.vacancy_id = c.vacancy_id AND b.skill < c.skill
GROUP BY 1, 2, 3 ORDER BY vacancies DESC, skill_1, skill_2, skill_3
```

Пары и тройки — самосоединение таблицы «вакансия–навык» с условием a.skill < b.skill (каждое сочетание считается один раз). Вывод запуска:

```
source: s3://hh-lake/dataset/it_vacancies_2024.csv
stats: [(7206, 4870, 1386)]
k=1: distinct=1376 total=32642
k=2: distinct=20802 total=129227
k=3: distinct=137413 total=411399
time=16.24s
```

4. Верификация: MapReduce против DuckDB

Скрипт 04_verify.py сравнивает оба набора файлов по каждому ключу: множества ключей, частоты, число зарплат, медианы (допуск 0,01 руб.) и md5 от отсортированного списка «ключ:частота:n_зарплат». Результат — полное совпадение на всех трёх размерах:

```
k=1: MR keys=1376 DuckDB keys=1376 sum MR=32642 sum DuckDB=32642
   only in MR=0 only in DuckDB=0 count mismatches=0 median mismatches=0
   md5 MR=5028d283831e80f676c178cfc3602f61 md5 DuckDB=5028d283831e80f676c178cfc3602f61 -> MATCH
   top: Java: MR=4484 DuckDB=4484
   top: Spring Framework: MR=2210 DuckDB=2210
   top: PostgreSQL: MR=1903 DuckDB=1903
   top: SQL: MR=1449 DuckDB=1449
   top: Git: MR=1179 DuckDB=1179
k=2: MR keys=20802 DuckDB keys=20802 sum MR=129227 sum DuckDB=129227
   only in MR=0 only in DuckDB=0 count mismatches=0 median mismatches=0
   md5 MR=7417abbb8009efa4dc7120d34cd1455c md5 DuckDB=7417abbb8009efa4dc7120d34cd1455c -> MATCH
   top: Java + Spring Framework: MR=2081 DuckDB=2081
   top: Java + PostgreSQL: MR=1773 DuckDB=1773
   top: Java + SQL: MR=1348 DuckDB=1348
   top: PostgreSQL + Spring Framework: MR=1159 DuckDB=1159
   top: Git + Java: MR=1091 DuckDB=1091
k=3: MR keys=137413 DuckDB keys=137413 sum MR=411399 sum DuckDB=411399
   only in MR=0 only in DuckDB=0 count mismatches=0 median mismatches=0
   md5 MR=fff609abd28886d217c7bd3e9936812a md5 DuckDB=fff609abd28886d217c7bd3e9936812a -> MATCH
   top: Java + PostgreSQL + Spring Framework: MR=1098 DuckDB=1098
   top: Git + Java + Spring Framework: MR=628 DuckDB=628
   top: Java + SQL + Spring Framework: MR=565 DuckDB=565
   top: Git + Java + PostgreSQL: MR=552 DuckDB=552
   top: Hibernate ORM + Java + Spring Framework: MR=546 DuckDB=546
RESULT: ALL MATCH
```

Расхождений нет: ни лишних ключей, ни разных частот, ни разных медиан. Совпадение подтверждает и то, что правило варианта одинаково реализовано в двух независимых средствах (Python re и RE2 в DuckDB).

5. Влияние навыков на заработную плату

Методика. Медианы взяты из результатов MapReduce (вакансии Java с навыками и зарплатой, n=1 074; медиана сегмента 193 200 руб.). Для пары (A,B) база — более частый навык A (по вакансиям с зарплатой), добавляемый — B: Δ = медиана(A и B) − медиана(A). Для тройки база — самая частая из трёх подпар, добавляемый — оставшийся навык: Δ = медиана(тройка) − медиана(базовая пара). Фильтр надёжности: пары n ≥ 15, тройки n ≥ 10 вакансий с зарплатой; в таблицах также число различных работодателей.

Важная находка о данных. Один работодатель (ID 6093775) выложил 396 Java-вакансий (148 с зарплатой), в основном «Java Developer/стажёр» в 20+ городах, с одной и той же ставкой 75 000 gross = 65 250 net. Это 11 % всех зарплат сегмента; они делают «65 250» медианой для любого навыка из его шаблона (Servlets, JSP, SVN, Hibernate, Oracle, MySQL…). Расчёт «на всех данных» поэтому отражает не навыки, а одного работодателя (в топе «понижающих» пар 10 из 10 имеют медиану ровно 65 250). Поэтому основная таблица ниже — без зарплат этого работодателя (--exclude-employers 6093775; частоты навыков не меняются), а результат на всех данных приведён для сравнения.

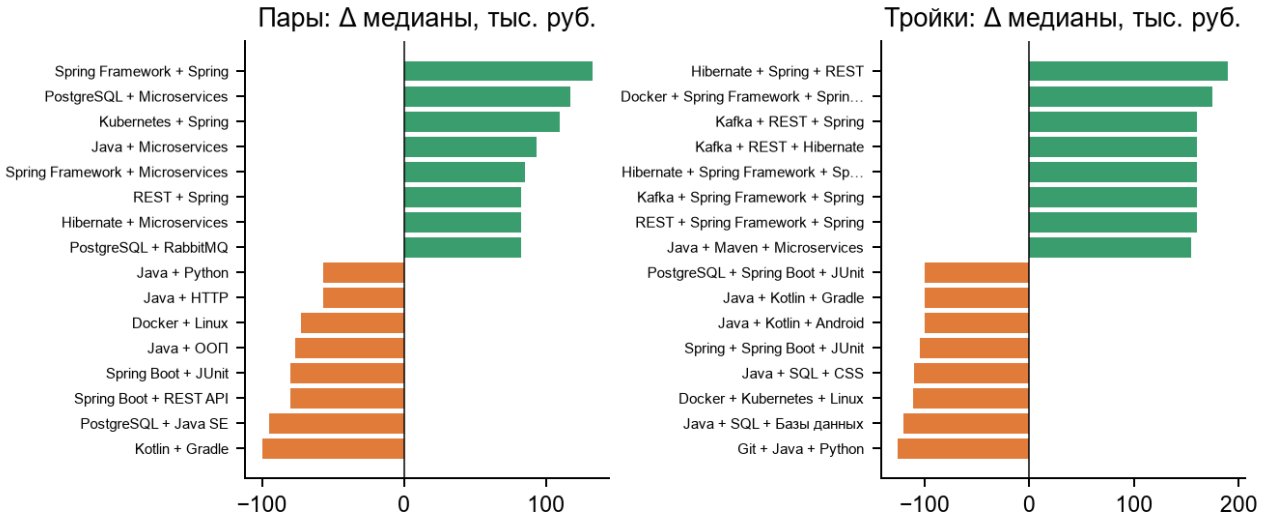


Рис. Δ медианы: 8 крупнейших прибавок и 8 крупнейших потерь (без массового работодателя).

Пары, повышающие медианную зарплату (без массового работодателя)

База (частый)	+ добавлен	n	работод.	Мед. базы	Мед. с навыком	Δ, руб.
Spring Framework	Spring	15	8	215 000	348 000	+133 000
PostgreSQL	Microservices	18	10	200 000	317 500	+117 500
Kubernetes	Spring	16	8	265 500	375 000	+109 500
Java	Microservices	30	17	206 875	300 000	+93 125
Spring Framework	Microservices	19	10	215 000	300 000	+85 000
REST	Spring	20	13	217 500	300 000	+82 500
Hibernate	Microservices	15	8	217 500	300 000	+82 500
PostgreSQL	RabbitMQ	50	22	200 000	282 462	+82 462

Пары, понижающие медианную зарплату

База (частый)	+ добавлен	n	работод.	Мед. базы	Мед. с навыком	Δ, руб.
Kotlin	Gradle	17	9	250 000	150 000	-100 000
PostgreSQL	Java SE	17	12	200 000	105 000	-95 000
Spring Boot	REST API	15	10	230 000	150 000	-80 000
Spring Boot	JUnit	23	10	230 000	150 000	-80 000
Java	ООП	64	40	206 875	130 250	-76 625
Docker	Linux	28	17	225 000	152 250	-72 750
Java	HTTP	17	10	206 875	150 000	-56 875
Java	Python	23	17	206 875	150 000	-56 875

Тройки, повышающие медианную зарплату

База (частый)	+ добавлен	n	работод.	Мед. базы	Мед. с навыком	Δ, руб.
Hibernate + Spring	REST	11	6	210 000	400 000	+190 000
Docker + Spring Framework	Spring	12	5	225 000	400 000	+175 000
Kafka + REST	Spring	13	7	240 000	400 000	+160 000
Kafka + REST	Hibernate	12	7	240 000	400 000	+160 000
Hibernate + Spring Framework	Spring	11	6	240 000	400 000	+160 000
Kafka + Spring Framework	Spring	11	5	240 000	400 000	+160 000
REST + Spring Framework	Spring	11	5	240 000	400 000	+160 000
Java + Maven	Microservices	10	5	220 000	374 100	+154 100

Тройки, понижающие медианную зарплату

База (частый)	+ добавлен	n	работод.	Мед. базы	Мед. с навыком	Δ, руб.
Git + Java	Python	10	6	200 000	74 000	-126 000
Java + SQL	Базы данных	12	8	200 000	80 040	-119 960
Docker + Kubernetes	Linux	11	8	261 000	150 000	-111 000
Java + SQL	CSS	11	6	200 000	90 000	-110 000
Spring + Spring Boot	JUnit	10	3	250 000	145 000	-105 000
Java + Kotlin	Android	10	9	250 000	150 000	-100 000
Java + Kotlin	Gradle	16	8	250 000	150 000	-100 000
PostgreSQL + Spring Boot	JUnit	17	8	250 000	150 000	-100 000

Для сравнения: те же расчёты на всех данных (с массовым работодателем)

```
[all] медиана сегмента Java (навыки и зарплата): 193200 руб., n=1074
[all] пар с n>=15: 361; троек с n>=10: 1340

=== [all] pairs: топ-10 ПОВЫШАЮТ ===
            base       added  n  employers  median_base  median_with    delta
           MySQL      Kotlin 17          1      65250.0     300000.0 234750.0
   Hibernate ORM       Redis 15          7      65250.0     280000.0 214750.0
   Hibernate ORM       Kafka 28         17      65250.0     250000.0 184750.0
           MySQL    RabbitMQ 25          4      65250.0     250000.0 184750.0
   Hibernate ORM Spring Boot 45         20      65250.0     240000.0 174750.0
   Hibernate ORM      Docker 35         16      65250.0     240000.0 174750.0
Spring Framework      Spring 15          8     179175.0     348000.0 168825.0
          ORACLE      Kotlin 19          7      65250.0     230000.0 164750.0
   Hibernate ORM        REST 44         20      65250.0     225600.0 160350.0
   Hibernate ORM       Linux 15          5      65250.0     225000.0 159750.0

=== [all] pairs: топ-10 ПОНИЖАЮТ ===
base         added   n  employers  median_base  median_with     delta
Java           JSP  46          1     190000.0      65250.0 -124750.0
Java Java Servlets  49          4     190000.0      65250.0 -124750.0
Java           SVN  55          3     190000.0      65250.0 -124750.0
Java  Unit Testing  72         15     190000.0      65250.0 -124750.0
Java       Java EE 162         16     190000.0      65250.0 -124750.0
Java     Java Core 182         23     190000.0      65250.0 -124750.0
Java        ORACLE 190         28     190000.0      65250.0 -124750.0
Java         MySQL 191         22     190000.0      65250.0 -124750.0
Java           ООП 209         41     190000.0      65250.0 -124750.0
Java Hibernate ORM 276         64     190000.0      65250.0 -124750.0

=== [all] triples: топ-10 ПОВЫШАЮТ ===
                base       added  n  employers  median_base  median_with    delta
Hibernate ORM + Java     Jenkins 10          5      65250.0     304500.0 239250.0
  MySQL + PostgreSQL    RabbitMQ 23          3      65250.0     300000.0 234750.0
  MySQL + PostgreSQL Spring Boot 23          4      65250.0     300000.0 234750.0
        Git + MySQL    RabbitMQ 23          3      65250.0     300000.0 234750.0
  MySQL + PostgreSQL      Docker 22          4      65250.0     300000.0 234750.0
```

Интерпретация

Что повышает. «Архитектурные» и высоконагрузочные навыки: Microservices (с Java: 300 000 против 206 875 у базы «Java», +93 тыс., n=30, 17 работодателей; с PostgreSQL: +117 тыс.), очереди RabbitMQ (с PostgreSQL: 282 462 против 200 000, n=50, 22 работодателя), Kubernetes, Kafka+REST. Это вакансии уровня senior/lead в банках и финтехе, где к Spring добавляют очереди, оркестрацию и микросервисы. Чтобы проверить, что это

не просто «больше опыта», я сравнил медианы внутри одной категории опыта (out/exp_check.txt): при опыте 3–6 лет вакансии с Microservices дают 300 000 против 243 600 без них, при 1–3 годах — 255 000 против 147 900; RabbitMQ при 3–6 годах — 290 000 против 239 625. Премия сохраняется внутри категории опыта, но n в ячейках невелик (7–14), поэтому это скорее направление эффекта, чем точная величина. Пары вида «Spring Framework + Spring» — синонимы одного навыка (разные формулировки в резюме работодателя), их высокая прибавка связана со стилем описания senior-вакансий, а не с отдельным навыком.

Что понижает. Базовые «учебные» и универсальные навыки: ООП (−76 625 к базе Java, n=64, 40 работодателей), HTML, JavaScript, Python, HTTP (−57 тыс.), тройки Java+Git+Python (−126 тыс.), Java+SQL+«Базы данных»/CSS (−110…−120 тыс.). Это вакансии начального уровня, фулстек-«на всё» и стажировки: у них короткий список требований из базовых тем, а зарплата ниже при тех же годах опыта (с ООП при 1–3 годах медиана 100 000 против 150 000, при 3–6 годах — 200 000 против 250 000; с HTML при 3–6 годах — 172 650 против 250 000). Пары Kotlin+Gradle, Spring Boot+JUnit/REST API, Docker+Linux понижают медиану относительно «сильной» базы: Android-/QA-/админские роли оплачиваются ниже, чем backend-разработка на Kotlin или Spring Boot.

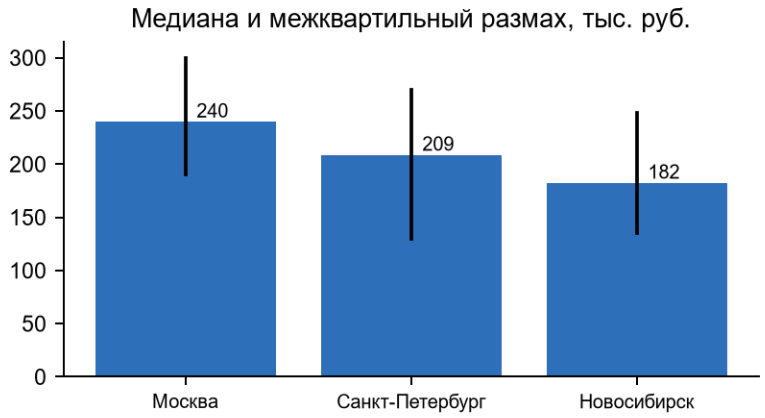
Ограничения. Зарплата указана лишь в ~19 % вакансий; в ячейках 10–50 вакансий и 3–20 работодателей, поэтому один работодатель может определять медиану; эффект навыка смешан с опытом и компанией — это корреляция, а не причинность; в данных один срез 2024 года.

6. Региональный анализ: Москва, Санкт-Петербург, Новосибирск

Выбраны крупнейшие по числу Java-вакансий центры: два столичных и один сибирский (разный масштаб рынка). Расчёт — DuckDB над теми же данными в бакете (06_regions.py). Зарплаты — руб./мес. на руки, только RUR, включая все вакансии с зарплатой (в том числе работодателя 6093775: он даёт 6 зарплат в Москве и 11 в СПб).

Таблица 1. Рынок и зарплаты

Регион	Вакансий	С навыками	% с нав.	С зарплатой	P25	Медиана	P75	Среднее	% удал.	% актив.
Москва	3642	2297	63.1	624	190 000	240 000	300 000	244 777	36.8	6.9
Санкт-Петербург	949	621	65.4	203	130 000	208 800	270 000	210 741	35.0	9.3
Новосибирск	305	209	68.5	72	135 000	182 500	248 463	184 979	31.8	11.5



Медиана зарплаты по опыту (n — число вакансий с зарплатой)

Опыт	Москва	Новосибирск	Санкт-Петербург
Нет опыта	75 000 (n=35)	65 250 (n=10)	65 250 (n=17)
От 1 года до 3 лет	182 438 (n=158)	145 000 (n=15)	127 500 (n=62)
От 3 до 6 лет	250 000 (n=373)	207 500 (n=44)	250 000 (n=101)
Более 6 лет	350 000 (n=58)	315 000 (n=3)	304 500 (n=23)

Таблица 2. Топ-10 навыков (доля вакансий региона с навыками; lift = доля в регионе / доля в сегменте; медиана зарплаты вакансий с навыком)

#	Москва	Санкт-Петербург	Новосибирск
1	Java: 90.8% (×0.99; 250k, n=420)	Java: 93.1% (×1.01; 200k, n=141)	Java: 95.2% (×1.03; 150k, n=59)
2	Spring Framework: 44.7% (×0.98; 250k, n=200)	Spring Framework: 47.7% (×1.05; 185k, n=83)	Spring Framework: 46.4% (×1.02; 150k, n=28)
3	PostgreSQL: 38.7% (×0.99; 250k, n=178)	PostgreSQL: 39.6% (×1.01; 168k, n=69)	PostgreSQL: 38.8% (×0.99; 168k, n=32)
4	SQL: 29.5% (×0.99; 241k, n=150)	Git: 26.4% (×1.09; 174k, n=59)	Git: 30.6% (×1.26; 178k, n=26)
5	Kafka: 24.2% (×1.19; 250k, n=95)	SQL: 25.9% (×0.87; 250k, n=48)	SQL: 30.6% (×1.03; 190k, n=22)
6	Spring Boot: 23.2% (×1.10; 240k, n=113)	Spring Boot: 21.6% (×1.02; 250k, n=40)	Hibernate ORM: 22.5% (×1.58; 168k, n=22)
7	Git: 20.8% (×0.86; 250k, n=120)	Kafka: 17.6% (×0.86; 250k, n=25)	Docker: 17.7% (×1.12; 150k, n=7)
8	Docker: 16.2% (×1.02; 250k, n=105)	Docker: 16.6% (×1.04; 250k, n=24)	Spring Boot: 14.8% (×0.70; 210k, n=11)
9	REST: 13.2% (×1.15; 250k, n=75)	Kotlin: 16.3% (×1.25; 300k, n=25)	ООП: 12.9% (×1.07; 65k, n=13)
10	Kotlin: 12.8% (×0.98; 255k, n=68)	Hibernate ORM: 12.6% (×0.89; 150k, n=33)	Kafka: 12.0% (×0.59; 178k, n=6)

Специфичные навыки региона (наибольший lift при ≥15 вакансий)

Москва	Санкт-Петербург	Новосибирск
Интеграционное тестирование (0.8%, ×1.91)	Mockito (3.1%, ×2.98)	QA (7.2%, ×1.63)
Camunda (2.1%, ×1.79)	JDBC (2.9%, ×2.02)	Hibernate ORM (22.5%, ×1.58)

Москва	Санкт-Петербург	Новосибирск
Allure (0.9%, ×1.77)	Управление командой (3.1%, ×1.91)	Linux (11.0%, ×1.54)
Apache Tomcat (1.8%, ×1.62)	Unit Testing (3.5%, ×1.50)	MySQL (8.1%, ×1.51)
Appium (0.7%, ×1.62)	Linux (10.6%, ×1.49)	ORACLE (11.5%, ×1.44)
Atlassian Confluence (1.6%, ×1.62)	JavaScript (6.0%, ×1.45)	Apache Maven (9.1%, ×1.39)
Hadoop (0.8%, ×1.61)	Atlassian Jira (3.7%, ×1.37)	Git (30.6%, ×1.26)

Самые частые пары и тройки по регионам (с медианой зарплаты)

Регион	Сочетание	Вакансий	С зарплатой	Медиана
Москва	Java + Spring Framework	957	190	250 000
Москва	Java + PostgreSQL	811	167	250 000
Москва	Java + SQL	625	131	247 950
Москва	Java + Kafka	510	85	250 000
Москва	Java + Spring Boot	503	109	247 950
Санкт-Петербург	Java + Spring Framework	277	74	185 000
Санкт-Петербург	Java + PostgreSQL	233	68	171 173
Санкт-Петербург	Git + Java	153	57	180 000
Санкт-Петербург	Java + SQL	150	46	250 000
Санкт-Петербург	PostgreSQL + Spring Framework	149	46	138 950
Новосибирск	Java + Spring Framework	95	28	150 000
Новосибирск	Java + PostgreSQL	78	31	180 000
Новосибирск	Git + Java	60	24	177 500
Новосибирск	Java + SQL	60	21	180 000
Новосибирск	PostgreSQL + Spring Framework	51	24	150 000

Регион	Сочетание	Вакансий	С зарплатой	Медиана
Москва	Java + PostgreSQL + Spring Framework	464	97	250 000
Москва	Java + SQL + Spring Framework	295	71	250 000
Москва	Java + Kafka + Spring Framework	292	42	250 000
Москва	Java + Kafka + PostgreSQL	267	50	250 000
Москва	Java + PostgreSQL + Spring Boot	256	55	250 000
Санкт-Петербург	Java + PostgreSQL + Spring Framework	143	45	130 000
Санкт-Петербург	Git + Java + Spring Framework	90	40	185 000
Санкт-Петербург	Java + PostgreSQL + Spring Boot	71	19	250 000
Санкт-Петербург	Git + Java + PostgreSQL	65	31	130 000
Санкт-Петербург	Java + SQL + Spring Framework	64	22	250 000
Новосибирск	Java + PostgreSQL + Spring Framework	50	24	150 000
Новосибирск	Git + Java + Spring Framework	41	19	150 000
Новосибирск	Hibernate ORM + Java + Spring Framework	41	18	120 000
Новосибирск	Git + Java + PostgreSQL	37	21	155 000
Новосибирск	Git + Java + SQL	31	13	210 000

Наблюдения (инсайты)

1. Москва платит больше и стабильнее. Медиана 240 000 против 208 800 в СПб (−13 %) и 182 500 в Новосибирске (−24 %). Нижний квартиль в Москве 190 000 против 130–135 тыс. — разница прежде всего «внизу»: в Москве почти нет дешёвых Java-вакансий. По опыту: 1–3 года — 182 тыс. (Москва) против 127 тыс. (СПб) и 145 тыс. (Новосибирск); у новичков (≈65–75 тыс.) регионы почти не различаются — там действует «стажёрская» ставка, в том числе из-за работодателя 6093775.

2. Ядро стека одинаково везде: Java, Spring Framework, PostgreSQL занимают первые три места в каждом регионе (39–47 % вакансий). Отличия — в «втором слое». Москва: Kafka (24,2 % и пятое место; ×1,19 к сегменту; в Новосибирске всего 12,0 %, ×0,59), Spring Boot (23,2 % против 14,8 %), специфичные Camunda (×1,79), Allure/Appium/интеграционное тестирование (автоматизация тестирования). Новосибирск — «классический enterprise»: Hibernate ORM (22,5 %, шестое место, ×1,58), Git, Oracle, MySQL, Maven; современного Kafka/Spring Boot мало. СПб: Kotlin (16,3 %, ×1,25, самая высокая медиана 300 000, n=25), Mockito, JDBC, Linux (×1,49), управление командой.

3. Один и тот же стек стоит по-разному. Тройка Java+PostgreSQL+Spring Framework — самая частая в каждом регионе, но её медиана 250 000 в Москве (n=97), 150 000 в Новосибирске (n=24) и всего 130 000 в СПб (n=45; на неё влияют стажёрские вакансии массового работодателя). Региональный фактор для этого набора сильнее, чем навык: в Москве Kafka (медиана 250 000) почти не добавляет к медиане региона (240 000) — премию даёт город, а не отдельный навык.

4. «Современный» и «классический» стек расходятся по цене внутри СПб: вакансии со Spring Boot — 250 000 (n=40), Kafka — 250 000 (n=25), Kotlin — 300 000, тогда как Spring Framework — 185 000 (n=83), PostgreSQL — 168 345, Hibernate ORM — 150 000 (n=33). Работодатели, пишущие «Spring Boot / Kotlin / Kafka», платят существенно больше тех, чьи вакансии построены на Spring Framework + Hibernate.

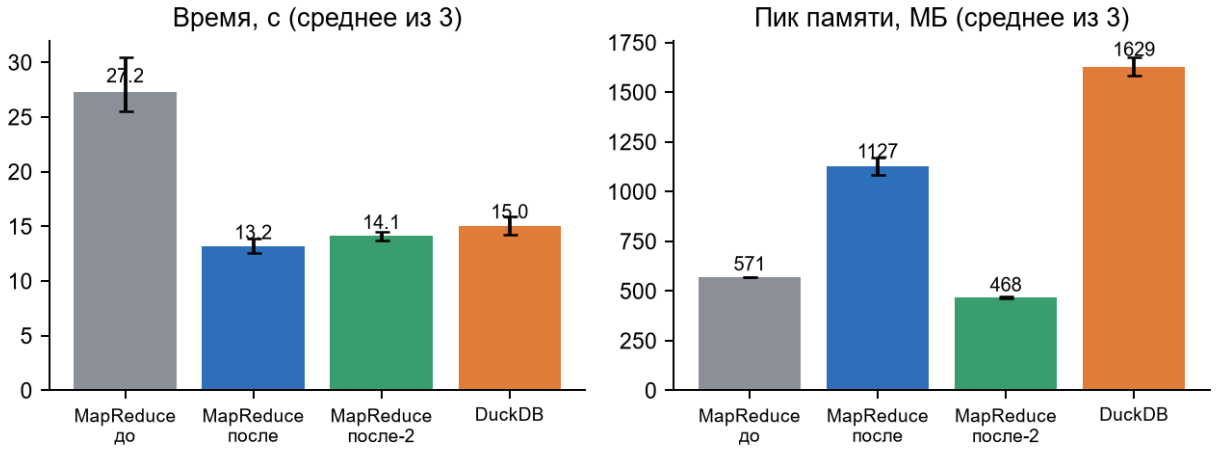
5. Структура рынка: доля удалёнки близка (36,8 / 35,0 / 31,8 %), доля вакансий с указанными навыками выше в малом регионе (68,5 % в Новосибирске против 63,1 % в Москве). Доля активных вакансий в снимке выше в регионах (11,5 % в Новосибирске против 6,9 % в Москве), что говорит о более медленном закрытии вакансий; в Новосибирске всего 72 вакансии с зарплатой, поэтому его цифры менее устойчивы.

7. Профилирование: время и память

Скрипт 07_profile.py запускает каждую конфигурацию 3 раза отдельным процессом и каждые 20 мс опрашивает суммарный RSS всего дерева процессов (главный процесс + дочерние mapper-ы), фиксируя время (wall-clock) и пик памяти. Все варианты читают данные из RustFS и считают одно и то же (навыки, пары, тройки, медианы); результат каждой оптимизации побайтно совпадает с исходным (md5 файлов). Лог запусков:

```
MapReduce до (naive: 1 процесс, чанк 64 МБ, без combiner) | run 1: 25.8 s, peak 570 MB, rc=0
MapReduce до (naive: 1 процесс, чанк 64 МБ, без combiner) | run 2: 25.5 s, peak 571 MB, rc=0
MapReduce до (naive: 1 процесс, чанк 64 МБ, без combiner) | run 3: 30.4 s, peak 571 MB, rc=0
MapReduce после (opt: 8 mapper-процессов, 8 потоков read-ahead, чанк 16 МБ, combiner) | run 1: 13.1 s, peak 1127 MB, rc=0
MapReduce после (opt: 8 mapper-процессов, 8 потоков read-ahead, чанк 16 МБ, combiner) | run 2: 12.5 s, peak 1170 MB, rc=0
MapReduce после (opt: 8 mapper-процессов, 8 потоков read-ahead, чанк 16 МБ, combiner) | run 3: 13.9 s, peak 1084 MB, rc=0
MapReduce после-2 (tuned: 4 mapper-процесса, 8 потоков read-ahead, чанк 8 МБ, combiner) | run 1: 13.7 s, peak 463 MB, rc=0
MapReduce после-2 (tuned: 4 mapper-процесса, 8 потоков read-ahead, чанк 8 МБ, combiner) | run 2: 14.5 s, peak 471 MB, rc=0
MapReduce после-2 (tuned: 4 mapper-процесса, 8 потоков read-ahead, чанк 8 МБ, combiner) | run 3: 14.1 s, peak 471 MB, rc=0
DuckDB (SQL к S3) | run 1: 15.9 s, peak 1631 MB, rc=0
DuckDB (SQL к S3) | run 2: 15.0 s, peak 1674 MB, rc=0
DuckDB (SQL к S3) | run 3: 14.2 s, peak 1582 MB, rc=0
[sweep] opt workers=8 io_threads=1 chunk=16MB: 30.7 s, peak 540 MB, rc=0
[sweep] opt workers=1 io_threads=8 chunk=16MB: 16.7 s, peak 510 MB, rc=0
[sweep] opt workers=4 io_threads=8 chunk=16MB: 13.2 s, peak 696 MB, rc=0
[sweep] opt workers=8 io_threads=4 chunk=16MB: 14.2 s, peak 772 MB, rc=0
[sweep] opt workers=8 io_threads=8 chunk=8MB: 14.4 s, peak 730 MB, rc=0
[sweep] opt workers=8 io_threads=8 chunk=32MB: 13.1 s, peak 1789 MB, rc=0
[sweep] opt workers=12 io_threads=8 chunk=16MB: 13.0 s, peak 1200 MB, rc=0
```

Конфигурация	Время 1/2/3, с	Среднее, с	Пик памяти 1/2/3, МБ	Среднее, МБ
MR до (1 проц., чанк 64 МБ)	25.8 / 25.5 / 30.4	27.2	570 / 571 / 571	571
MR после (8 проц., чанк 16 МБ)	13.1 / 12.5 / 13.9	13.2	1127 / 1170 / 1084	1127
MR после-2 (4 проц., чанк 8 МБ)	13.7 / 14.5 / 14.1	14.1	463 / 471 / 471	468
DuckDB (SQL к S3)	15.9 / 15.0 / 14.2	15.0	1631 / 1674 / 1582	1629



Сравнение подходов

MapReduce против DuckDB. Исходный MapReduce — 27.2 с и 571 МБ, DuckDB — 15.0 с и 1629 МБ. DuckDB быстрее (в 1.8 раза), потому что читает файл параллельно несколькими потоками на C++, разбирает CSV векторизованно и не передаёт данные между процессами; но он тратит в 2.9 раза больше памяти (буферы параллельного разбора CSV, хеш-таблицы самосоединений — тройки материализуются как 411 тыс. строк в соединении). Простой MapReduce держит в памяти один чанк и накапливает только счётчики, поэтому экономнее, но однопоточный: ограничен последовательным чтением по сети и интерпретатором Python. После оптимизации MapReduce (14.1 с) сравнялся с DuckDB по времени (15.0 с), оставаясь в 3.5 раза экономнее по памяти (468 против 1629 МБ); зато DuckDB проще — один SQL-запрос вместо сотни строк кода.

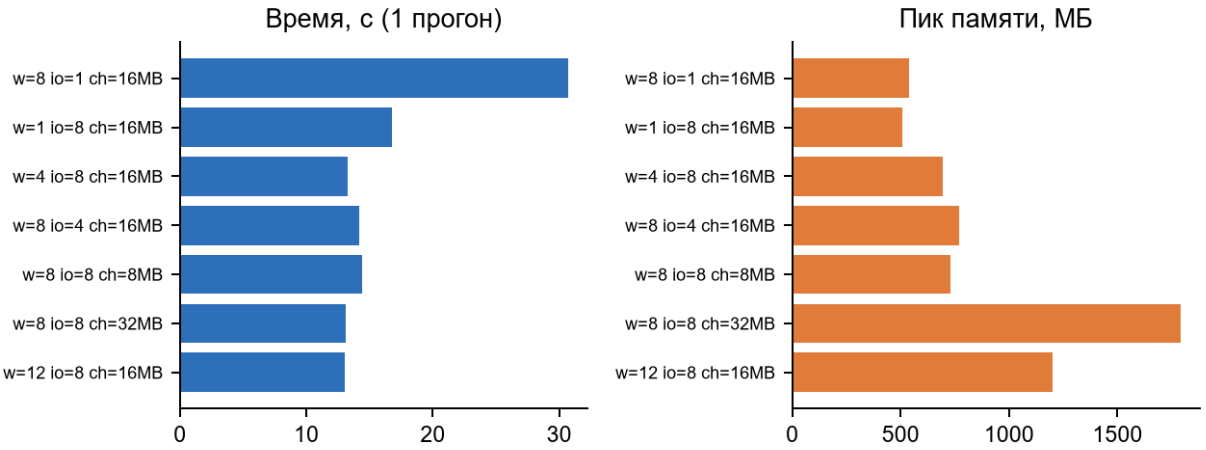
Бонус: оптимизация MapReduce — «до» и «после»

Профиль показал, что в исходной версии узкое место — не разбор, а последовательное чтение из хранилища: чтение всего объекта Range-запросами в один поток занимает ≈19–21 с из ≈25 с, разбор 64 МБ CSV — менее секунды. Применены оптимизации: (1) read-ahead — окно из 8 параллельных Range-запросов (чтение ускорилось до ≈8 с); (2) пул mapper-процессов с ограниченным окном задач (разбор идёт

параллельно с чтением); (3) combiner в mapper-е; (4) подбор размера чанка и числа процессов по памяти.

	Время, с	Пик памяти, МБ	Ускорение	Память
До (1 процесс, чанк 64 МБ)	27.2	571	1,0×	1,0×
После (8 процессов, чанк 16 МБ)	13.2	1127	2.1×	2.0×
После-2 (4 процесса, чанк 8 МБ)	14.1	468	1.9×	0.82×

Первая оптимизированная конфигурация даёт ускорение 2.1×, но потребляет в 2.0 раза больше памяти (8 процессов × 16 МБ блоков и их копии). Настроенная версия (4 процесса, чанк 8 МБ) сохраняет ускорение 1.9× и при этом использует меньше памяти, чем исходная (468 против 571 МБ). Ниже — однократные запуски для разбора вклада каждого фактора:



Выводы по графику: при io=1 (без read-ahead) 8 процессов не ускоряют работу — всё упирается в одно соединение с хранилищем; при w=1, io=8 уже достигнуто 16,7 с (чтение параллельно, разбор последовательно); с 4 процессами достигается плато (≈13 с) — дальше упираемся в предел пропускной способности хранилища на локальной машине и накладные расходы передачи блоков между процессами; рост числа процессов до 12 и чанка до 32 МБ лишь увеличивает память (до 1,2–1,8 ГБ).

8. Итоги

• RustFS развёрнут, бакет hh-lake содержит исходный CSV и результаты обоих способов.
• Собственный MapReduce (чтение чанками из S3, повтор упавшего mapper-а) и SQL-запрос DuckDB дают идентичные частоты навыков, пар и троек и медианы (1 376 / 20 802 / 137 413 ключей); сумма частот совпадает с Σ C(n,k).
• Зарплату повышают микросервисы и очереди сообщений, понижают базовые универсальные навыки (ООП, HTML, JavaScript, Python); результат устойчив при контроле опыта; выявлено искажение массовым работодателем и рассчитан вариант без него.
• Москва — самая высокая и стабильная оплата (медиана 240 тыс.), СПб — выделяется Kotlin/Spring Boot, Новосибирск — классический стек с Hibernate/Oracle.
• Оптимизация MapReduce: 1.9× по времени (27 → 14 с) и −18 % по памяти (571 → 468 МБ); оптимизированный MapReduce сравнялся с DuckDB по времени (15 с) и использует в 3.5 раза меньше памяти.